AlgebraKing: A Computer Algebra System for the Terminal and for Claude.

User Guide, Version 0.0.1

Universidade Federal de Goiás

LEEDMOL - Laboratory of Electronic Structure and Molecular Dynamics.



https://github.com/phfmatias/AlgebraKing

Contents

1 What it is **1**

2 Installation **1**

2.1 Requirements 1
2.2 Automatic installation 1
2.3 Manual installation 2
2.4 Updating 2
2.5 Uninstalling 2

3 Getting started **2**

3.1 The four ways to use it 2
3.2 The REPL 3
3.3 `;` shows, `:` silences 3
3.4 `%`, `%%`, `%%%` 4
3.5 2-D or 1-D 4
3.6 `.mpl` scripts 4
3.7 Exit code 4
3.8 Ready-made examples 4

4 Language **5**

4.1 Assignment, unassignment, evaluation 5
4.2 Procedures and arrow functions 5
4.3 Operator derivative `D`, composition `@` and `@@` 6
4.4 Control flow 6
4.5 `seq`, `add`, `mul`, `map`, `select`, `zip`, lists 6
4.6 Matrix and Vector with angle brackets 7
4.7 Packages: `with`, `:-` and `[]` 8
4.8 `read` 8
4.9 `try` / `catch` 8
4.10 `assume`, `additionally`, `assuming`, `about` 9
4.11 `Digits` and `evalf[n]` 9
4.12 `printf`, `sprintf`, `print`, `lprint`, `latex` 9
4.13 Precedence and operators 10

5 Mathematics **10**

5.1 Algebra and simplification 10
5.2 Polynomials 11
5.3 Number theory and integer functions 11
5.4 Calculus 12

5.5 Solving equations . . . 13
5.6 Linear algebra (`LinearAlgebra`) . . . 13
5.7 Transforms (`inttrans`) . . . 14
5.8 Vector calculus (`VectorCalculus`) . . . 15
5.9 Elementary and special functions . . . 15
5.10 Miscellaneous: types and structure . . . 16

6 Using with Claude (MCP) 16
6.1 Automatic registration . . . 16
6.2 Manual registration . . . 16
6.3 Claude Desktop . . . 17
6.4 The 8 tools . . . 17
6.5 How the verdict works . . . 22
6.6 Sessions . . . 23
6.7 Environment variables . . . 23

7 Known limitations 24

8 Troubleshooting 25

9 Validation 26

Complete guide to AlgebraKing. All the examples below were run on the `algebraking` binary (version 0.0.1) and the output shown is the real output, in 1-D format (the format used by scripts, by `-e` when the output is not a terminal, and by `interface(prettyprint=0)`; see "Getting started").

1 What it is

AlgebraKing is a computer algebra system for the terminal, **inspired by the Maple language**. It was written from scratch, in **clean room**: no Maple code, library (`.mla`), help text or data was used; the language comes only from public documentation and general knowledge of it. It reproduced 257/257 verdicts of a real verification suite originally run in Maple 18.

The architecture has two halves:

- **Rust** handles the *language*: lexer, parser, evaluation (variables, procedures, loops, `%`, packages, `with`), 1-D and 2-D printing and the MCP server.
- **Python + SymPy** handles the *mathematics*: every math function (`int`, `solve`, `dsolve`, `Eigenvalues`, …) is a call to an engine (`engine/engine.py`) embedded in the binary, which translates the AlgebraKing functions to SymPy and returns the result in the AlgebraKing convention.

Why does it exist? The main use is *checking calculations*: Claude takes a step of your derivation (derivative, integral, ODE solution, identity) and asks AlgebraKing, via MCP, whether it is correct, receiving `equal`, `not_equal` (with a counterexample) or `undetermined`. The second use is a terminal REPL. When the program does not know how to do something, it returns the expression unevaluated or raises an error; the rule is never to return a wrong answer with confidence.

Out of scope: GUI, worksheets (`.mw`), plot window (`plot`) and code generation, or byte-for-byte identical formatting to any other system.

---

2 Installation

2.1 Requirements

Item	Version	Note
Linux (tested on Ubuntu)	any recent	
Python 3	≥ 3.8	the engine runs on it
`sympy` and `mpmath`	SymPy 1.14 (tested)	`pip install --user sympy mpmath`
Rust / `cargo`	recent stable	`install.sh` installs it via rustup if missing
`claude` (Claude Code)	optional	only to register the MCP server

2.2 Automatic installation

```
git clone https://github.com/phfmatias/AlgebraKing.git
cd AlgebraKing
./install.sh
```

The script is idempotent (it can be run again without problems) and does, in this order:

1. checks for `python3 >= 3.8` and `import sympy, mpmath`; if missing, runs `python3 -m pip install --user sympy mpmath` (if pip refuses with "externally managed environment", it retries with `--break-system-packages`, only in the user directory);
2. checks for `cargo`; if missing, installs rustup (minimal profile, no prompts) in `~/.cargo`;
3. `cargo build --release`;
4. creates the symbolic link `~/.local/bin/algebraking` pointing to `target/release/algebraking` (warns, with the exact line for `~/.bashrc`, if `~/.local/bin` is not in `PATH`);
5. runs a self-test: `algebraking -e 'int(x^2,x);'` must print `x^3/3`;
6. registers the MCP server in Claude Code, if the `claude` command exists (and skips if `claude mcp get algebraking` already answers);
7. prints a summary with the next steps.

Flag	Effect
`--no-mcp`	does not register the MCP server in Claude Code
`--no-python-deps`	does not check or install `sympy`/`mpmath`
`--prefix DIR`	symbolic link directory (default: `~/.local/bin`)
`-h, --help`	help

2.3 Manual installation

```
python3 -m pip install --user sympy mpmath
cargo build --release
mkdir -p ~/.local/bin
ln -sfn "$PWD/target/release/algebraking" ~/.local/bin/algebraking
export PATH="$HOME/.local/bin:$PATH"          # coloque no ~/.bashrc
algebraking -e 'int(x^2,x);'                   # x^3/3
```

2.4 Updating

```
git pull && ./install.sh
```

2.5 Uninstalling

```
./uninstall.sh
```

Removes the symbolic link, the MCP registration (`claude mcp remove algebraking -s user`) and the engine cache (`~/.cache/algebraking`). It **never** deletes the repository folder; delete it by hand if you want. The REPL history (`~/.algebraking_history`) is also kept.

---

3 Getting started

3.1 The four ways to use it

```
algebraking                    sessão interativa (REPL)
algebraking arquivo.mpl        executa um script
algebraking -e "código"        avalia uma string
algebraking mcp                servidor MCP no stdin/stdout
```

Also: `algebraking -V` (version) and `algebraking -h` (help).

3.2 The REPL

The prompt is `>`. A statement can span several lines: while it is incomplete, the prompt becomes `.` `Ctrl-C` discards the pending input, `Ctrl-D` (or `quit;`) exits. The history (arrow keys, `Ctrl-R`) is saved in `~/.algebraking_history`.

```
$ algebraking
AlgebraKing 0.0.1 - symbolic math (SymPy engine). Type quit; to leave.
> f := x -> x^2*exp(-x):
> int(f(x), x=0..infinity);
2
> diff(f(x), x);
   2  -x        -x
- x ·e   + 2·x·e
> A := <<1|2>, <3|4>>:
> with(LinearAlgebra):
> MatrixInverse(A);
⎡-2    1  ⎤
⎢          ⎥
⎣3/2  -1/2⎦
> sqrt(2)/(1+x)^2;
   √2
────────
       2
(x + 1)
> %;
   √2
────────
       2
(x + 1)
> interface(prettyprint=0):
> diff(f(x), x);
-x^2*exp(-x)+2*x*exp(-x)
```

3.3 `;` shows, `:` silences

`;` shows the result and `:` suppresses it (the last statement may omit the terminator):

```
> a := 5; b := a^2: c := a + b;
a := 5
c := 30
```

(Real output of `algebraking -e 'a := 5; b := a^2: c := a + b;'`.)

3.4 `%`, `%%`, `%%%`

`%` is the last result shown, `%%` the one before it, `%%%` the one before that:

```
x^2 + y; %; %%;
x^2+y
x^2+y
x^2+y
```

3.5 2-D or 1-D

In the REPL, and in scripts/`-e` when the output **is a terminal**, results are printed in 2-D (as above). When the output goes to a pipe or file, they are printed in 1-D. To force it: `interface(prettyprint=0)` (1-D) or `interface(prettyprint=1)` (2-D).

3.6 `.mpl` scripts

A script is a text file with AlgebraKing statements. The program prints the results of the statements ending in `;` and **stops at the first error, with exit code 1**.

```
$ cat stop.mpl
a := 2;
b := a^10;
error "parou aqui";
c := 3;
$ algebraking stop.mpl; echo "rc=$?"
a := 2
b := 1024
Error, parou aqui
rc=1
```

3.7 Exit code

Code	Meaning
`0`	all good (or `quit` called in the script)
`1`	evaluation or syntax error in script/`-e`; unreadable file; engine fails to start
`2`	incorrect command-line usage (unknown option)

3.8 Ready-made examples

The `examples/` folder has `basics.mpl`, `calculus.mpl`, `linear_algebra.mpl`, `ode.mpl` and `procedures.mpl`:

```
$ algebraking examples/calculus.mpl
f := x -> x^2*exp(-x)
x -> -x^2*exp(-x)+2*x*exp(-x)
exp(-1)
```

```
-sin(x)^2+cos(x)^2
1
arctan(x)
exp(1)
1+x+x^2/2+x^3/6+x^4/24+O(x^5)
Pi^2/6
x-x^2/2+x^3/3+O(x^4)
```

---

4 Language

All the examples in this section: `algebraking -e '...'`, 1-D output.

4.1 Assignment, unassignment, evaluation

Input	Output
`a := 5; b := a^2; a := 'a'; b;`	`a := 5b := 25a := a25`
`x := 3; x := 'x'; x;`	`x := 3x := xx`
`x := 5: 'x'; evaln(x); unassign('x'): x;`	`xxx`
`a, b := 1, 2; a+b;`	`a, b := 1, 23`
`restart; x;`	`x`
`x + x; 2*(x+1);`	`2*x2*x+2`

`restart` clears variables, assumptions, packages and `Digits`. Single quotes (`'x'`) suspend evaluation; `x := 'x'` unassigns. Names with spaces use backticks: `` `a b` ``.

4.2 Procedures and arrow functions

```
f := x -> x^2 + 1; f(3); g := (x,y) -> x*y; g(2,5);
f := x -> x^2+1
10
g := (x, y) -> x*y
10
```

```
p := proc(n) local k, s; s := 0; for k from 1 to n do s := s + k end do;
 ↪  return s end proc; p(10);
p := proc(n) local k, s; s := 0; for k from 1 to n do s := s+k end do;
 ↪  return s end proc
55
```

`proc` accepts `local`, `global`, `option remember`, typed parameters (`n::nonnegint`), `args`/`nargs`, `return`, `error` and closes with `end proc` (or `end`):

```
fib := proc(n::nonnegint) option remember; if n < 2 then n else fib(n-1) +
 ↪  fib(n-2) end if end proc: fib(50);
12586269025
```

```
f := proc() nargs, [args] end proc: f(a, b, c);
3, [a, b, c]
```

`unapply` turns an expression into a function:

```
h := unapply(x^2+y, x); h(3);
h := x -> x^2+y
y+9
```

4.3 Operator derivative `D`, composition `@` and `@@`

```
f := x -> x^3; D(f); D(f)(2); (D@@2)(f);
f := x -> x^3
x -> 3*x^2
12
x -> 6*x

f := (x,y) -> x^2*y; D[1](f); D[2](f);
f := (x, y) -> x^2*y
(x, y) -> 2*x*y
(x, y) -> x^2

(sin@cos)(x);
sin(cos(x))
```

4.4 Control flow

```
for i from 10 to 1 by -3 do printf("%d ", i) end do; printf("\n");
10 7 4 1

for x in [1,2,3] do printf("%d ", x^2) end do; printf("\n");
1 4 9

if 3 > 2 then "yes" else "no" end if;
"yes"
```

`while`, `break`, `next`, `elif`, and the short forms `fi` and `od` also work. As in The statements of a top-level loop ending in `;` show the result:

```
for i to 3 do i; end do;
1
2
3
```

4.5 `seq`, `add`, `mul`, `map`, `select`, `zip`, lists

Input	Output
`seq(i^2, i=1..6);`	`1, 4, 9, 16, 25, 36`
`add(i, i=1..100); mul(i, i=1..6);`	`5050720`
`map(x -> x^2, [1,2,3]);`	`[1, 4, 9]`

Input	Output
`select(isprime, [seq(i,i=1..20)]);`	`[2, 3, 5, 7, 11, 13, 17, 19]`
`zip((a,b)->a+b, [1,2,3],[4,5,6]);`	`[5, 7, 9]`
`L := [3,1,2]; sort(L); nops(L); op(2, L); L[2]; subsop(2=9, L);`	`L := [3, 1, 2][1, 2, 3]311[3, 9, 2]`
`{3,1,2} union {5}; {1,2} intersect {2,3}; member(2, [1,2,3]);`	`{1, 2, 3, 5}{2}true`
`x$3; y(i)$i=1..3;`	`x, x, xy(1), y(2), y(3)`
`f~([1,2,3]); sin~([0, Pi/2]);`	`[f(1), f(2), f(3)][0, 1]`
`sum(i^2, i=1..10); add(i^2, i=1..10);`	`385385`

With symbolic bounds, `add`/`mul` switch to the engine's `sum`/`product`. `eval(e, x=a)`, `subs(x=a, e)` (this argument order) and `subsop` also exist:

```
eval(x^2+y, x=2); subs(x=2, x^2+y); eval(x^2+y, [x=1,y=2]);
y+4
y+4
3
```

Tables and strings: `t := table(): t[a] := 1: t[b] := 2: t[a]+t[b];` gives `3`; `length("abc")` gives `3`; `cat("a", 1)` gives `"a1"`.

4.6 Matrix and Vector with angle brackets

Syntax	Result
`<1,2,3>`	`Vector[column]([1, 2, 3])`
`<1|2|3>`	`Vector[row]([1, 2, 3])`
`<<1|2>,<3|4>>`	`Matrix(2, 2, [[1, 2], [3, 4]])`
`Matrix(2,3,(i,j)->i+j)`	`Matrix(2, 3, [[2, 3, 4], [3, 4, 5]])`
`Vector[row]([1,2])`	`Vector[row]([1, 2])`

```
A := <<1|2>,<3|4>>; A . A; A^(-1); A^2;
A := Matrix(2, 2, [[1, 2], [3, 4]])
Matrix(2, 2, [[7, 10], [15, 22]])
Matrix(2, 2, [[-2, 1], [3/2, -1/2]])
Matrix(2, 2, [[7, 10], [15, 22]])

M := <<1|2>,<3|4>>: map(x->x^2, M); 2*M + M; M . <1,2>;
Matrix(2, 2, [[1, 4], [9, 16]])
Matrix(2, 2, [[3, 6], [9, 12]])
Vector[column]([5, 11])
```

Matrix product is the dot `.`; `M[1,2]` indexes.

4.7 Packages: `with`, `:-` and `[]`

There are three ways to call a package function, and the short name only works **after `with`**:

```
LinearAlgebra:-Determinant(<<1|2>,<3|4>>);
-2

LinearAlgebra[Transpose](<<1|2>,<3|4>>);
Matrix(2, 2, [[1, 3], [2, 4]])

with(LinearAlgebra): Determinant(<<1|2>,<3|4>>);
-2

Determinant(<<1|2>,<3|4>>);
Determinant(Matrix(2, 2, [[1, 2], [3, 4]]))
```

(The last one does not resolve the short name and returns the call unevaluated.) Recognized packages: `LinearAlgebra`, `inttrans`, `VectorCalculus`, `student`, `numtheory`, `Groebner`, `combinat`, `ListTools`, `DEtools`. When two packages export the same name (for example `Basis`, in `LinearAlgebra` and `Groebner`), the last one loaded with `with` wins.

4.8 `read`

`read "arquivo.mpl"` executes a file (path relative to the current directory):

```
$ cat lib.mpl
sq := x -> x^2:
cube := x -> x^3:
printf("loaded\n"):
$ algebraking -e 'read "lib.mpl"; sq(3) + cube(2);'
loaded
17
```

4.9 `try` / `catch`

```
try 1/0; catch: print("caught"); end try;
"caught"

try error "boom" catch "boom": print("caught boom") end try;
"caught boom"

try error "boom %1", 5 catch: lprint(lasterror) end try;
"boom 5"

r := proc(x) try if x < 0 then error "negative: %1", x end if; sqrt(x)
 ↪  catch: -1 finally print("done") end try end proc; r(4); r(-4);
r := proc(x) try if x < 0 then error "negative: %1", x end if; sqrt(x)
 ↪  catch: -1 finally print("done") end try end proc
"done"
2
"done"
-1
```

Errors outside `try` appear as `Error, message`; in the REPL the session continues, in a script the

program stops.

4.10 `assume`, `additionally`, `assuming`, `about`

```
assume(x>0): x; x^2; about(x);
x
x^2
x:
  is assumed to be: 0 < x

sqrt(x^2) assuming x>0;
x

sqrt(x^2) assuming x::real;
abs(x)

is(x>0) assuming x>1;
true

assume(n, integer): sin(n*Pi); additionally(n>0): about(n);
0
n:
  is assumed to be: n::integer, 0 < n

simplify(sqrt(x^2), assume=positive);
x
```

4.11 `Digits` and `evalf[n]`

```
evalf(Pi); evalf[30](Pi); Digits := 20; evalf(sqrt(2));
3.141592654
3.14159265358979323846264338328
Digits := 20
1.4142135623730950488

evalf(Pi, 50);
3.14159265358979323846264338327950288419716939937751
```

`Digits` is 10 by default; `evalf(e, n)` and `evalf[n](e)` evaluate `e` with `Digits = n`.

4.12 `printf`, `sprintf`, `print`, `lprint`, `latex`

```
printf("%d! = %d\n", 20, 20!); printf("%a is %f\n", x, 3.14159);
↪  printf("%.3e\n", 12345.678);
20! = 2432902008176640000
x is 3.141590
1.235e+04

sprintf("%5.2f|%-4d|%s", 3.14159, 42, "ok");
" 3.14|42  |ok"

latex(x^2/3);
\frac{x^{2}}{3}
```

Specifiers: `%d %a %f %e %g %s`. `kernelopts(version)` returns `"AlgebraKing 0.0.1"`.

4.13 Precedence and operators

Operator precedence: `||`, `$`, `!`, `^`/`**` (**non-associative**), unary `-`, `* /`, `+ -`, `mod`, `..`, relations, `not`, `and`, `or`, `xor`, `implies`, `->`, `,`, `:=`.

```
a^b^c;
Error, ambiguous use of `^`, please use parentheses

1/3 + 1/6; 2^100; 30!; 7 mod 3; 10!/(8!);
1/2
1267650600228229401496703205376
265252859812191058636308480000000
1
90
```

---

5 Mathematics

This section lists what actually exists in the engine, by area, with one executed example. Any name that belongs neither to the language nor to the engine comes back unevaluated (`undefined_fn(3)` → `undefined_fn(3)`). Results are given in 1-D.

5.1 Algebra and simplification

Functions: `simplify` (also `simplify(e, trig)`, `assume=`), `expand`, `factor` (with extensions), `normal`, `combine`, `collect`, `sort`, `rationalize`, `radnormal`, `evalc`, `evalf`, `evalb`, `is`, `testeq`, `convert`.

Input	Output
`expand((x+1)^3);`	`x^3+3*x^2+3*x+1`
`factor(x^3-6*x^2+11*x-6);`	`(x-3)*(x-2)*(x-1)`
`factor(x^2+1, I);`	`(x-I)*(x+I)`
`simplify(sin(x)^2+cos(x)^2);`	`1`
`normal((x^2-1)/(x-1));`	`x+1`
`combine(sin(x)*cos(x), trig);`	`sin(2*x)/2`
`collect(x*y+x*z+y, x);`	`(y+z)*x+y`
`sort(x+x^3+x^2+1, x);`	`x^3+x^2+x+1`
`rationalize(1/(1+sqrt(2)));`	`sqrt(2)-1`
`radnormal(sqrt(8));`	`2*sqrt(2)`
`evalc(exp(I*Pi/3));`	`sqrt(3)*I/2+1/2`
`convert((x^2+1)/(x*(x-1)), parfrac, x);`	`2/(x-1)-1/x+1`
`convert(sin(x), exp);`	`-I*(exp(I*x)-exp(-I*x))/2`
`testeq(sin(x)^2+cos(x)^2, 1);`	`true`
`is(sqrt(2) > 1);`	`true`

Input	Output
`evalb(2+2=4);`	`true`

`convert` accepts `parfrac`, `exp`, `trig`, `sincos`, `tan`, `ln`, `polynom`, `rational`, `list`, `set`, `float`, `radical`, `piecewise`, `Heaviside`, `binomial`, `factorial`, `GAMMA` (examples: `convert(0.75, rational)` → `3/4`; `convert([1,2,3], set)` → `{1, 2, 3}`).

5.2 Polynomials

Functions: `coeff`, `coeffs`, `degree`, `ldegree`, `lcoeff`, `tcoeff`, `numer`, `denom`, `quo`, `rem`, `gcd`, `lcm`, `resultant`, `discrim`, `content`, `primpart`, `sqrfree`, `roots`, `RootOf`/`allvalues`, `interp`, `Groebner[Basis]`.

Input	Output
`coeff(3*x^2+2*x+1, x, 2); degree(x^5+x, x);`	`35`
`coeffs(x^2+3*x+1, x);`	`1, 3, 1`
`quo(x^3+1, x+1, x); rem(x^3+2, x+1, x);`	`x^2-x+11`
`gcd(x^2-1, x^2+2*x+1); lcm(x^2-1, x+3);`	`x+1x^3+3*x^2-x-3`
`resultant(x^2-1, x-2, x);`	`3`
`discrim(a*x^2+b*x+c, x);`	`-4*a*c+b^2`
`sqrfree(x^3-x^2-x+1);`	`[1, [[x+1, 1], [x-1, 2]]]`
`content(6*x^2+9*x, x); primpart(6*x^2+9*x, x);`	`32*x^2+3*x`
`roots(x^2-5*x+6);`	`[[3, 1], [2, 1]]`
`interp([1,2,3],[1,4,9], x);`	`x^2`
`Groebner[Basis]([x^2+y, x*y-1], plex(x,y));`	`[y^2+x, y^3+1]`
`allvalues(RootOf(x^2-2, x));`	`sqrt(2), -sqrt(2)`

5.3 Number theory and integer functions

Functions: `ifactor`, `ifactors`, `isprime`, `ithprime`, `nextprime`, `prevprime`, `igcd`, `ilcm`, `iquo`, `irem`, `isqrt`, `iroot`, `mod`, `modp`, `mods`, `numtheory[phi]`, `sigma`, `tau`, `mobius`, `divisors`, `binomial`, `factorial`, `floor`, `ceil`, `round`, `trunc`, `frac`, `abs`, `signum`, `max`, `min`, `fibonacci`, `lucas`, `bernoulli`, `euler`, `harmonic`, `pochhammer`, `doublefactorial`.

Input	Output
`ifactor(360); ifactors(360);`	`2^3*3^2*5[1, [[2, 3], [3, 2], [5, 1]]]`
`isprime(97); ithprime(10);` `nextprime(100); prevprime(100);`	`true2910197`
`igcd(12, 18); ilcm(4, 6);` `iquo(17, 5); irem(17, 5);`	`61232`

`numtheory[phi](36);`
`numtheory[sigma](12);`
`numtheory[tau](12);`
`numtheory[mobius](30);`
	`12286-1`

Input	Output
`numtheory[divisors](12);`	`{1, 2, 3, 4, 6, 12}`
`binomial(10,3); factorial(6);`	`120720`
`floor(2.7); ceil(2.1); round(2.5); trunc(-2.7); frac(2.75);`	`233-20.75`
`max(1,5,3); min(4,2); isqrt(50); iroot(27, 3);`	`5273`
`with(combinat): fibonacci(10);`	`55`
`lucas(5); bernoulli(4); euler(4); harmonic(5);`	`11-1/305137/60`
`pochhammer(3, 2); doublefactorial(5);`	`1215`

Note: `fibonacci` belongs to `combinat` (`combinat[fibonacci](10)` or `with(combinat);` without that the call comes back unevaluated). After `with(numtheory)`, `phi(36)` and `divisors(12)` work without a prefix.

5.4 Calculus

Functions: `diff`, `D`, `int`, `Int`, `value`, `limit`, `series`, `taylor`, `mtaylor`, `sum`, `product`, `Sum`, `Product`, `Limit`, `Diff`, `residue`, `singular`, `implicitdiff`, `extrema`, `maximize`, `minimize`.

Input	Output
`diff(sin(x)*x, x);`	`x*cos(x)+sin(x)`
`diff(x^4, x$2);`	`12*x^2`
`diff(x^2*y^3, x, y);`	`6*x*y^2`
`int(x^2, x);`	`x^3/3`
`int(x^2, x=0..3);`	`9`
`int(exp(-x^2), x=-infinity..infinity);`	`sqrt(Pi)`
`int(sin(x)^2, x=0..Pi);`	`Pi/2`
`Int(x^2,x); value(Int(x^2,x));`	`Int(x^2, x)x^3/3`
`limit(sin(x)/x, x=0);`	`1`
`limit(1/x, x=0, right);`	`infinity`
`limit(abs(x)/x, x=0, left);`	`-1`
`limit((1+1/n)^n, n=infinity);`	`exp(1)`
`series(exp(x), x=0, 5);`	`1+x+x^2/2+x^3/6+x^4/24+O(x^5)`
`taylor(cos(x), x=0, 6);`	`1-x^2/2+x^4/24+O(x^6)`
`mtaylor(sin(x)*cos(y), [x,y], 4);`	`-x^3/6-x*y^2/2+x`
`sum(1/k^2, k=1..infinity);`	`Pi^2/6`
`sum(k, k=1..n);`	`n^2/2+n/2`
`product(k, k=1..5);`	`120`
`residue(1/(x^2+1), x=I);`	`-I/2`
`singular(1/(x*(x-1)));`	`{x = 0, x = 1}`
`implicitdiff(x^2+y^2=1, y, x);`	`-x/y`

Input	Output
`extrema(x^3-3*x, {}, x);`	`{-2, 2}`
`maximize(-x^2+2*x, x); minimize(x^2-2*x, x);`	`1-1`

5.5 Solving equations

Functions: `solve`, `fsolve`, `isolve`, `rsolve`, `dsolve`, `pdsolve`, `odetest`.

Input	Output
`solve(x^2-5*x+6=0, x);`	`3, 2`
`solve({x+y=3, x-y=1}, {x,y});`	`{x = 2, y = 1}`
`solve(x^3=1, x);`	`1, sqrt(3)*I/2-1/2, -sqrt(3)*I/2-1/2`
`solve(x^2>4, x);`	`RealRange(-infinity, Open(-2)), RealRange(Open(2), infinity)`
`solve(a*x^2+b*x+c=0, x);`	`(-b-sqrt(-4*a*c+b^2))/(2*a), (-b+sqrt(-4*a*c+b^2))/(2*a)`
`fsolve(cos(x)=x, x);`	`0.7390851332`
`isolve(2*x+3*y=7);`	`{x = 3*_Z1-7, y = -2*_Z1+7}`
`rsolve({f(n+1)=2*f(n), f(0)=1}, f(n));`	`2^n`
`dsolve(diff(y(x),x)=-2*y(x), y(x));`	`y(x) = _C1*exp(-2*x)`
`dsolve({diff(y(x),x,x)+y(x)=0, y(0)=0, D(y)(0)=1}, y(x));`	`y(x) = sin(x)`
`dsolve({diff(x(t),t)=y(t), diff(y(t),t)=-x(t)}, {x(t),y(t)});`	`{x(t) = _C1*sin(t)+_C2*cos(t), y(t) = _C1*cos(t)-_C2*sin(t)}`
`odetest(y(x)=exp(-2*x)*_C1, diff(y(x),x)=-2*y(x));`	`0`
`pdsolve(diff(u(x,t),x)+diff(u(x,t),t)=0, u(x,t));`	`u(x, t) = _F1(-t+x)`

5.6 Linear algebra (`LinearAlgebra`)

Functions (use `with(LinearAlgebra)` or the `LinearAlgebra:-` prefix): `Matrix`, `Vector`, `Determinant`, `MatrixInverse`, `Transpose`, `HermitianTranspose`, `Eigenvalues`, `Eigenvectors`, `Rank`, `Trace`, `NullSpace`, `ColumnSpace`, `RowSpace`, `LinearSolve`, `ReducedRowEchelonForm`, `GaussianElimina` `LUDecomposition`, `QRDecomposition`, `CharacteristicPolynomial`, `MinimalPolynomial`, `JordanForm`, `DotProduct`, `CrossProduct`, `Norm`, `Normalize`, `IdentityMatrix`, `ZeroMatrix`, `DiagonalMatrix`, `MatrixExponential`, `MatrixPower`, `IsDefinite`, `GramSchmidt`, `SingularValues`.
Arithmetic: `A.B`, `A^(-1)`, `A+B`, `k*A`.

With `A := <<2|1>,<1|3>>` and `with(LinearAlgebra):`

Input	Output
`Determinant(A);`	`5`

Input	Output
`MatrixInverse(A);`	`Matrix(2, 2, [[3/5, -1/5], [-1/5, 2/5]])`
`Trace(A); Rank(A);`	`52`
`Eigenvalues(A);`	`Vector[column]([-sqrt(5)/2+5/2, sqrt(5)/2+5/2])`
`Eigenvectors(A);`	`Vector[column]([-sqrt(5)/2+5/2, sqrt(5)/2+5/2]), Matrix(2, 2, [[-sqrt(5)/2-1/2, sqrt(5)/2-1/2], [1, 1]])`
`Eigenvalues(evalf(A));`	`Vector[column]([1.38196601125011, 3.61803398874989])`

With `B := <<1|2>,<3|4>>`:

Input	Output
`CharacteristicPolynomial(B, x);`	`x^2-5*x-2`
`LinearSolve(B, <5,6>);`	`Vector[column]([-4, 9/2])`
`ReducedRowEchelonForm(B);`	`Matrix(2, 2, [[1, 0], [0, 1]])`
`QRDecomposition(B);`	`Matrix(2, 2, [[sqrt(10)/10, 3*sqrt(10)/10], [3*sqrt(10)/10, -sqrt(10)/10]]), Matrix(2, 2, [[sqrt(10), 7*sqrt(10)/5], [0, sqrt(10)/5]])`
`NullSpace(<<1|2>,<2|4>>);`	`{Vector[column]([-2, 1])}`
`MatrixExponential(<<0|1>,<0|0>>, t);`	`Matrix(2, 2, [[1, t], [0, 1]])`
`JordanForm(<<4|1>,<2|3>>);`	`Matrix(2, 2, [[2, 0], [0, 5]])`
`DotProduct(<1,2,3>,<4,5,6>);`	`32`
`CrossProduct(<1,0,0>,<0,1,0>);`	`Vector[column]([0, 0, 1])`
`Normalize(<3,4>, 2);`	`Vector[column]([3/5, 4/5])`
`GramSchmidt([<1,1>,<1,0>]);`	`[Vector[column]([1, 1]), Vector[column]([1/2, -1/2])]`
`SingularValues(<<3|0>,<0|4>>);`	`Vector[column]([4, 3])`
`IsDefinite(<<2|0>,<0|3>>, query='positive_definite');`	`true`

Floating-point matrices use a numerical solver (mpmath) with at least 15 digits.

5.7 Transforms (`inttrans`)

Functions: `laplace`, `invlaplace`, `fourier`, `invfourier`, `mellin` (with `with(inttrans)`):

Input	Output
`laplace(sin(t), t, s);`	`1/(s^2+1)`
`invlaplace(1/(s^2+1), s, t);`	`sin(t)`
`fourier(exp(-t^2), t, w);`	`sqrt(Pi)*exp(-w^2/4)`
`invfourier(exp(-w^2/4)*sqrt(Pi), w, t);`	`exp(-t^2)`
`mellin(exp(-x), x, s);`	`GAMMA(s)`

5.8 Vector calculus (`VectorCalculus`)

Functions: `Gradient`, `Divergence`, `Curl`, `Laplacian`, `Jacobian`, `Hessian`, `Wronskian` (with `with(VectorCalculus)`):

Input	Output
`Gradient(x^2+y^2, [x,y]);`	`Vector[column]([2*x, 2*y])`
`Divergence(<x^2,y^2>, [x,y]);`	`2*x+2*y`
`Curl(<-y,x,0>, [x,y,z]);`	`Vector[column]([0, 0, 2])`
`Laplacian(x^2+y^2, [x,y]);`	`4`
`Jacobian([x*y, x+y], [x,y]);`	`Matrix(2, 2, [[y, x], [1, 1]])`
`Hessian(x^2*y, [x,y]);`	`Matrix(2, 2, [[2*y, 2*x], [2*x, 0]])`
`Wronskian([sin(x),cos(x)], x);`	`Matrix(2, 2, [[sin(x), cos(x)], [cos(x), -sin(x)]])`

5.9 Elementary and special functions

Elementary: `sin cos tan cot sec csc`, `sinh ... csch`, `arcsin ... arccsc`, `arcsinh ... arccsch`, `exp`, `ln`, `log[b]`, `log10`, `log2`, `sqrt`, `root`, `surd`, `abs`, `signum`, `csgn`, `argument`, `Re`, `Im`, `conjugate`.

Special: `GAMMA`, `Beta`, `Psi`, `Zeta`, `erf`, `erfc`, `erfi`, `erfinv`, `Ei`, `Si`, `Ci`, `Shi`, `Chi`, `Li`, `FresnelS`, `FresnelC`, `LambertW`, `BesselJ/Y/I/K`, `AiryAi/Bi`, `LegendreP`, `HermiteH`, `LaguerreL`, `ChebyshevT/U`, `GegenbauerC`, `JacobiP`, `hypergeom`, `EllipticK/E/F/Pi`, `dilog`, `polylog`, `Heaviside`, `Dirac`, `piecewise`.

Input	Output
`GAMMA(1/2); Beta(2,3); Psi(1); Zeta(2);`	`sqrt(Pi)1/12-gammaPi^2/6`
`erf(1.0); Ei(1.0); Si(1.0); LambertW(1.0);`	`0.84270079291.8951178160.94608307040.5671432904`
`BesselJ(0, 1.0); AiryAi(0.5);`	`0.76519768660.2316936065`
`LegendreP(2, x); HermiteH(3, x); LaguerreL(2, x);`	`3*x^2/2-1/28*x^3-12*xx^2/2-2*x+1`
`ChebyshevT(3, x); ChebyshevU(2, x);`	`4*x^3-3*x4*x^2-1`
`EllipticK(0.5); EllipticE(0.5);`	`1.6857503551.467462209`

Input	Output
`dilog(2.0); polylog(2, 0.5);`	`-0.82246703340.5822405265`
`arcsin(1/2); arctan(1, 1); log[2](8); log10(1000);`	`Pi/6Pi/433`
`sqrt(18); surd(-8, 3); root(16, 4);`	`3*sqrt(2)-22`
`csgn(-1+I); argument(1+I); Re(3+4*I); conjugate(3+4*I);`	`-1Pi/433-4*I`
`Heaviside(-1);`	`0`
`piecewise(x<0, -x, x);`	`piecewise(x < 0, -x, x)`

`hypergeom([1,2],[3],x)` comes back unevaluated (it only evaluates in certain cases).

5.10 Miscellaneous: types and structure

`type`, `whattype`, `has`, `indets`, `lhs`, `rhs`, `op`, `nops`, `subs`, `eval`, `unapply`.

Input	Output
`type(3, integer); type(x, polynom); type([1,2], list);`	`truetruetrue`
`whattype(2.5);`	`float`
`indets(x^2+y*z); has(x^2+y, y);`	`{x, y, z}true`
`lhs(a=b); rhs(a=b);`	`ab`
`op(x+y); nops(x+y+z);`	`x, y3`
`op(0, f(x,y));`	`f`

---

6 Using with Claude (MCP)

MCP (Model Context Protocol) is the protocol through which an assistant such as Claude calls external tools. `algebraking mcp` is an MCP server that talks over stdin/stdout (JSON-RPC 2.0): Claude sends the step of your calculation and receives the verdict.

6.1 Automatic registration

`./install.sh` already does the registration (if the `claude` command exists). To redo it:

```
./install.sh
```

6.2 Manual registration

```
claude mcp add algebraking -s user -- "$(command -v algebraking)" mcp
```

(the `-s user` scope applies to all your projects). To verify:

```
claude mcp list              # algebraking deve aparecer como conectado
claude mcp get algebraking   # mostra comando, argumentos e estado
```

Inside Claude Code, the `/mcp` command shows the servers and tools. Sessions that are already open must be restarted to load a newly registered server.

6.3 Claude Desktop

Edit `~/.config/Claude/claude_desktop_config.json` (Linux) and use the **absolute** path of the binary (JSON does not expand ~):

```
{
  "mcpServers": {
    "algebraking": {
      "command": "/home/SEU_USUARIO/.local/bin/algebraking",
      "args": ["mcp"]
    }
  }
}
```

Restart Claude Desktop after saving.

6.4 The 8 tools

All of them accept AlgebraKing syntax. Arguments with `?` are optional. `session?` selects a named session (default `"main"`). All the checking tools accept `field?`: `real` (default, because physics and chemistry calculations use real variables) or `complex`.

Tool	Arguments	What it does
`algebra_eval`	`code`, `session?`	runs AlgebraKing statements in a persistent session and returns each result in 1-D, plus the LaTeX of the last one
`verify_equal`	`lhs`, `rhs`, `assumptions?`, `field?`, `session?`	decides whether `lhs = rhs` on the common domain: verdict, method, conditions, domain of each side, difference, counterexample inside the domain, hypothesis suggestion and limitations
`check_derivative`	`f`, `derivative`, `var` (+ `assumptions?`, `field?`, `session?`)	checks whether `diff(f, var)` is the stated derivative
`check_integral`	`integrand`, `antiderivative`, `var` (+ `assumptions?`, `field?`, `session?`)	checks whether `diff(antiderivative, var)` is the integrand (correct antiderivative up to a constant)

Tool	Arguments	What it does
`check_solution`	`equations`, `solution` (+ `assumptions?`, `field?`, `session?`)	checks each candidate in the **original** equation and its domain, lists the rejected ones and says whether the accepted ones are all the solutions
`check_step`	`before`, `after`, `var?` (+ `assumptions?`, `field?`, `session?`)	classifies a solution step: equivalent, implies (spurious solutions), is implied (lost solutions), not equivalent or undetermined
`check_ode_solution`	`ode`, `solution`, `func` (+ `assumptions?`, `field?`, `session?`)	like `odetest`: substitutes into the ODE and the initial conditions
`algebra_reset`	`session?`	`restart` of the session

`assumptions` is a string such as `"x>0, n::integer"`. In `check_solution`, `equations` can be an equation or a set (`{x+y=3, x-y=1}`); an expression `e` means `e = 0`; `solution` is one candidate (`x=2`, `{x=2, y=1}`) or several (`[x=0, x=3]`, `[{x=1, y=2}, {x=2, y=1}]`). In `check_ode_solution`, the ODE can come in a set with initial conditions.

6.4.1 Response fields

The first line is always `verdict: equal | not_equal | undetermined`, but the verdict alone is not enough. Also read:

Field	Meaning
`method`	`symbolic` (proof), `numeric` (sampling), `convergence`, `solveset`, `domain`
`field`	field used: `real` or `complex`
`conditions`	existence conditions of the common domain, in AlgebraKing syntax (`x <> 1`, `abs(x) < 1`, `sin(x) <> 0`). `equal` holds **only** where they hold
`domain`	domain of each side, the common one, and whether both sides have the same natural domain (`same domain: no, they differ where x = 1`)
`counterexample`	only appears with a point **inside** the common domain, with the values of each side
`outside the common real domain`	point where one side is real and in its domain, and the other is not real (for example `2*ln(x)` with `x < 0`)
`suggestion`	hypothesis under which the equality was **proved symbolically** (`equal if x >= 0`); does not change the verdict

Field	Meaning
`note` / `limitation`	explanations and what was not proved (sampling, convergence, sets that `solveset` did not solve)
`assumptions used`	the session hypotheses and those from the `assumptions` argument

`sum`, `product` and `int` reach the engine **inert** (`Sum`, `Product`, `Int`) in the checking tools, so that the convergence condition is not lost; in `algebra_eval` and the REPL they keep returning the closed form.

6.4.2 Examples of request and real response

`verify_equal` — *"Check whether (x^2-1)/(x-1) equals x+1."*

```
verdict: equal
method: symbolic
field: real
conditions: x <> 1 (the result holds only where these hold)
domain: lhs: x <> 1; rhs: all real numbers; common: x <> 1; same domain:
↪  no, they differ where x = 1
difference lhs - rhs: 0
note: equal where both sides are defined; the domains differ where x = 1
lhs: (x^2-1)/(x-1)
rhs: x+1
```

"sqrt(x^2) = x?" — without an assumption the counterexample is inside the domain, and the tool suggests the hypothesis that makes the identity true:

```
verdict: not_equal
method: numeric
field: real
conditions: none
domain: lhs: all real numbers; rhs: all real numbers; common: all real
↪  numbers; same domain: yes
difference lhs - rhs: -x+abs(x)
counterexample: x = -2.1 (inside the common domain); lhs = 2.1, rhs = -2.1
suggestion: equal if x >= 0 -- found by trying a fixed menu of assumptions;
↪  the equality under it is a symbolic proof
note: the counterexample lies inside the common domain
```

With `assumptions: "x>0"` the result is `equal`, method `symbolic`, with `assumptions used: x > 0`.

"ln(x^2) = 2 ln(x)?" — on the common domain (`x > 0`) the sides coincide, but for `x < 0` the left side is real and the right one is not:

```
verdict: not_equal
method: numeric
field: real
conditions: x > 0 (the result holds only where these hold)
```

```
domain: lhs: x <> 0; rhs: x > 0; common: x > 0; same domain: no, they differ
 ↪  where x < 0
difference lhs - rhs: -2*ln(x)+ln(x^2)
outside the common real domain: at x = -2.1 lhs = 1.483874689, rhs =
 ↪  1.483874689+6.283185307*I
suggestion: equal if x > 0 -- found by trying a fixed menu of assumptions;
 ↪  the equality under it is a symbolic proof
note: the sides agree on the common real domain, but at x = -2.1 the lhs is
 ↪  real and in its domain while the rhs is not real (outside its real
 ↪  domain) and has a different value
note: over the reals the rhs requires x > 0, the lhs only x <> 0
```

The valid identity is `ln(x^2) = 2*ln(abs(x))`: it gives `equal`, `conditions: x <> 0`, `same domain: yes`.

"sum(x^n, n=0..infinity) = 1/(1-x)?" — the series only converges for `abs(x) < 1`:

```
verdict: equal
method: symbolic
field: real
conditions: abs(x) < 1 (the result holds only where these hold)
domain: lhs: abs(x) < 1; rhs: x <> 1; common: abs(x) < 1; same domain: no,
 ↪  they differ where x <= -1 or x > 1
difference lhs - rhs: 0
note: Sum(x^n, n = 0 .. infinity) = 1/(1-x) only where abs(x) < 1
 ↪  (convergence condition)
note: Sum(x^n, n = 0 .. infinity) diverges at x = -1 (boundary), x = 1
 ↪  (boundary), x = -2, x = 2
note: equal where both sides are defined; the domains differ where x <= -1 or
 ↪  x > 1
limitation: divergence outside abs(x) < 1 was checked only at the listed
 ↪  points, not proved
lhs: Sum(x^n, n = 0 .. infinity)
rhs: 1/(-x+1)
```

Trigonometric identities with poles carry the condition: `(1-cos(x)^2)/sin(x)^2 = 1` gives `equal` with `conditions: sin(x) <> 0` and the note `sin(x) <> 0: excludes x = Pi*_Z1 (_Z1 integer)` (`_Z1` is an arbitrary integer).

`check_step` — *"Check the step: from sqrt(x+1) = x-1 I squared both sides and arrived at x+1 = (x-1)^2."* (`before: "sqrt(x+1) = x-1"`, `after: "x+1 = (x-1)^2"`)

```
classification: implies (every solution of before solves after, but after
 ↪  has extra (spurious) solutions)
solutions before: x = 3
solutions after: x = 0, x = 3
spurious solutions (added by the step): x = 0
verdict: not_equal
method: solveset
field: real
conditions: x >= -1 (domain of the equations)
domain: before: x >= -1; after: all real numbers
note: the new equation is the old one squared (lhs^2 = rhs^2); squaring
 ↪  adds the solutions of lhs = -rhs, so those must be rejected by checking
 ↪  them in the original equation
before: sqrt(x+1) = x-1
after: x+1 = (x-1)^2
```

Dividing by the unknown loses solutions (`x^2 = x` for `x = 1`):

```
classification: implied_by (every solution of after solves before, but
↪  solutions of before are lost)
solutions before: x = 0, x = 1
solutions after: x = 1
lost solutions: x = 0
...
note: the step divides by x, which vanishes at solutions of the original
↪  equation; those solutions are lost
```

Infinite families appear as families, never as finite lists: from `sin(x) = 0` to `x = 0` the result is `implied_by` with `solutions before: x = Pi*_Z1 (_Z1 integer)` and `lost solutions: x = Pi*_Z1 (_Z1 integer) except x = 0`. When `solveset` does not solve one of the equations (it returns `ConditionSet`, as in `exp(x) = x+2`), the classification is `undetermined`, with the reason. With two expressions (no =), the step is a rewrite and is checked as in `verify_equal`, with domains. Parameters other than `var` are treated as generic by `solveset`; the report warns about it, and a factor made only of parameters appears in the note (`a*x = a*b` for `x = b` loses every `x` when `a = 0`).

`check_derivative` — *"Check whether the derivative of x*exp(x) is exp(x)."*

```
The claimed derivative is WRONG.
computed diff(f, x): x*exp(x)+exp(x)
verdict: not_equal
method: numeric
field: real
conditions: none
domain: lhs: all real numbers; rhs: all real numbers; common: all real
↪  numbers; same domain: yes
difference lhs - rhs: x*exp(x)
counterexample: x = -2.1 (inside the common domain); lhs = -0.1347020711,
↪  rhs = 0.1224564283
note: the counterexample lies inside the common domain
```

And with `(x+1)*exp(x)`: `The claimed derivative is correct.` (`verdict: equal`, `method: symbolic`).

`check_integral` — *"Check whether an antiderivative of x*cos(x) is x*sin(x)+cos(x)."*

```
The antiderivative is correct (up to an additive constant).
diff(antiderivative, x): x*cos(x)
verdict: equal
method: symbolic
field: real
conditions: none
domain: lhs: all real numbers; rhs: all real numbers; common: all real
↪  numbers; same domain: yes
difference lhs - rhs: 0
```

`check_solution` — *"Check whether x=0 and x=3 solve sqrt(x+1) = x-1."* (`equations: "sqrt(x+1) = x-1"`, `solution: "[x=0, x=3]"`)

```
Some candidates are REJECTED: x = 0.
equation 1: sqrt(x+1) = x-1
```

```
candidate x = 0: rejected
  equation 1: not satisfied: lhs = 1, rhs = -1
candidate x = 3: accepted
  equation 1: satisfied: lhs - rhs simplifies to 0
all solutions over the reals: x = 3
completeness: the accepted candidates are all the solutions over the reals
verdict: not_equal
method: symbolic
field: real
conditions: x >= -1 (domain of the equations)
```

A candidate outside the domain is rejected even if the calculation "works out" after simplifying: `x = 1` in `(x^2-1)/(x-1) = 2` gives `outside domain: the condition x-1 <> 0 of the original equation fails.` For `sin(x) = 0` with `[x=0, x=Pi]`, both are accepted, but the completeness line says `the equation has infinitely many solutions over the reals (x = Pi*_Z1 (_Z1 integer)); a finite list of candidates cannot be complete.` System `{x+y=3, x-y=1}` with `{x=2, y=1}`: `The solution satisfies every equation.` and `completeness: the accepted candidates are all the solutions of the polynomial system.`

`check_ode_solution` — *"Check whether y(x)=3 sin(x)+2 cos(x) solves y' '+y=0."* (`ode: "diff(y(x),x,x)+y(x)=0", solution: "y(x)=3*sin(x)+2*cos(x)", func: "y(x)"`)

```
The solution satisfies the ODE (residual 0).
condition 1: diff(y(x), x, x)+y(x) = 0
  after substitution: 0 = 0
  verdict: equal
  method: symbolic
  field: real
  conditions: none
  domain: lhs: all real numbers; rhs: all real numbers; common: all real
  ↪  numbers; same domain: yes
  difference lhs - rhs: 0
```

`algebra_eval` — *"Compute the derivative of x^2 sin(x) in AlgebraKing."* (`code: "f := x^2*sin(x): diff(f, x);"`)

```
x^2*cos(x)+2*x*sin(x)
LaTeX: x^{2} \cos{\left(x \right)} + 2 x \sin{\left(x \right)}
```

`algebra_reset` — *"Clear the session."* → `Session reset.`

6.5 How the verdict works

`verify_equal` (and the tools `check_derivative`, `check_integral`, `check_ode_solution` and `check_step` with two expressions, which use it underneath) follows this procedure, over the requested field (`real` by default):

1. For lists, sets, equations and matrices, it compares element by element (no domain analysis; the report warns about it).
2. `Sum`, `Product` and `Int` are computed keeping the SymPy condition (`abs(x) < 1`); convergence at the boundary and at two points outside is tested and reported. A divergent series has no value; a series whose convergence cannot be decided never gives `equal`.
3. It collects the existence conditions of each side: denominators `<> 0`, argument of `ln` (`> 0` over

the reals, `<> 0` over the complexes), roots and fractional powers (`>= 0` over the reals), poles of `tan`, `cot`, `sec`, `csc`, ranges of `arcsin`, `arccos`, `arctanh`, etc. Conditions already guaranteed by the hypotheses disappear. The excluded sets come from `solveset`; when it cannot solve, the condition stays symbolic and a limitation warns about it.

4. It tries to prove `lhs - rhs = 0` symbolically on the common domain (simple conditions, such as `x > 0`, become hypotheses). Zero → **`equal`**, method **`symbolic`**.
5. Numerical test: random points at 40 digits, **only** where both sides exist (and are real, in the real field) and satisfy the conditions. A clearly different point → **`not_equal`** with that point as the counterexample. A point where one side is real and the other is not → **`not_equal`** explained (`outside the common real domain`). All equal → numeric `equal`.
6. `equal` means equal on the common domain; if the domains differ (`x = 1` excluded only on the left, for example), the report says where.
7. If the verdict is not `equal` and there are at most 2 variables, it tries a fixed menu of hypotheses (`x >= 0`, `x > 0`, `x < 0`, `x::integer`) and, if any gives a symbolic proof, shows `suggestion`. The main verdict does not change.
8. Otherwise, **`undetermined`**. The program never raises an error for a mathematical reason.

What is proof and what is heuristic. Proof: `equal` with method `symbolic`, under the listed `conditions` and the declared hypotheses; the `suggestion` is also a symbolic proof under the suggested hypothesis, but the choice of hypothesis is heuristic (a fixed menu, not a complete search). Heuristic: numeric `equal` (sample points), divergence outside the condition of a series (tested at points), the domain comparison with two or more variables, and the classification of `check_step` when there are parameters (`solveset` treats them as generic). The `check_step` and the completeness of `check_solution` depend on `solveset`: when it returns `ConditionSet`, the answer is `undetermined` / *not established*, never a guess. Finally, the language automatically simplifies `x/x` to `1` and `sqrt(x)^2` to `x`; the tool recovers these conditions by reading the input before evaluation, but not when the expression comes from a variable already assigned in the session.

Numeric `equal` is strong evidence, not proof. Real example: `erf(x)+erfc(x) = 1` gives `equal` by method `numeric`, and that is correct. But `sin(x)*Heaviside(x)` against `abs(sin(x))*Heaviside(x` also gives numeric `equal`, because the test points lie in [-3, 3] and the difference only shows up for x > π (at `x = 4` the two expressions are `sin(4)` and `-sin(4)`). Be wary of identities with piecewise-defined functions, branches or behavior in distant regions. `symbolic` + `equal` is the only answer that counts as a demonstration.

The language's `verify(a, b)` function (REPL and scripts) still returns only `true`, `false` or `FAIL`, using the old algorithm (no field or domain).

6.6 Sessions

Each server process keeps named sessions, each with its own interpreter: variables, `Digits`, assumptions and loaded packages persist between calls in the same session. Without `session`, `"main"` is used. Two sessions do not see each other:

```
algebra_eval  code="g := 3;"  session="s2"   ->  g := 3
algebra_eval  code="g + 1;"   session="s2"   ->  4
```

`algebra_reset` clears a session. Closing Claude Code ends the server and discards the sessions.

6.7 Environment variables

Variable	Effect
`ALGEBRAKING_TIMEOUT`	seconds per engine call (default 60). On timeout: the engine is killed, restarted and the error is `Error, (in name) computation timed out after N s`; the session's variables and assumptions are kept
`ALGEBRAKING_PYTHON`	engine Python executable (default `python3`); useful if SymPy is in another Python/venv
`ALGEBRAKING_DEBUG`	any value: forwards the engine's `stderr` (`[engine] ...` lines) and prints Python tracebacks of internal failures

In the MCP registration, pass variables with `claude mcp add algebraking -s user -e ALGEBRAKING_TIMEOUT=120 -- ...` or in the `env` key of the Claude Desktop JSON.

---

7 Known limitations

An honest list of what to expect. The goal is a correct answer first; output formatting is secondary.

Topic	Behavior
`solve`	returns all the solutions it finds, including those from several branches: `solve(sin(x)=0, x)` → `Pi, 0`
Order of solutions	`solve(x^2-5*x+6=0, x)` → `3, 2`
Order of terms and factors	comes from SymPy's internal form: `(x-3)*(x-2)*(x-1)`, `-sqrt(5)/2+5/2`
Floating-point format	`0.1362`; `1.5*10^20`
Names with assumptions	`assume(x>0)` does not mark the name: `x` appears as `x`
`Heaviside(0)`	`1/2`
Last digits	15-digit floating-point results (numeric matrices, `HFloat`) may differ in the last two digits from other systems

Functions and packages that **do not exist** (the call comes back unevaluated or gives a syntax error): `inttrans:-hilbert`, `LegendreQ`, the `Student` package (`Student[Calculus1]`), `isolate` (comes back unevaluated), `DEtools` graphics. There is also **no**: graphics (`plot`, `plots`), graphical interface, `.mw` worksheets, code generation and the `Statistics`/`StringTools` libraries (the package name is recognized, but without functions).

The 1-D output is not byte-for-byte identical to any other system's.

---

8 Troubleshooting

`sympy/mpmath` not found. Typical symptom:

```
Error, (in int) the math engine stopped:   File ".../engine-....py", line
↪  29, in <module> |     import mpmath | ModuleNotFoundError: No module
↪  named 'mpmath'
```

Install: `python3 -m pip install --user sympy mpmath` (or run `./install.sh`). If SymPy is in a Python different from the `python3` in `PATH`, point to it with `ALGEBRAKING_PYTHON=/path/python3`. Do not pass `-I` or `-s` to Python: SymPy lives in the user's `site-packages`.

Engine does not start. Wrong `ALGEBRAKING_PYTHON`:

```
Error, cannot start the math engine (/nao/existe): No such file or directory
↪  (os error 2)
```

`cargo: command not found`. Run `./install.sh` (it installs rustup) or load the environment with `source ~/.cargo/env` and check `cargo --version`.

`algebraking: command not found`. `~/.local/bin` is not in `PATH`. Add it to `~/.bashrc` and open a new terminal:

```
export PATH="$HOME/.local/bin:$PATH"
```

MCP server shows as `failed` / does not connect.

1. `claude mcp get algebraking`: check the registered command (absolute path of the binary
   - `mcp`) and that the file exists (`ls -l` on the path; the link breaks if the repository moved somewhere else).
2. Test the server by hand (it should answer with one JSON line containing `serverInfo`):

   ```
   printf '%s\n' '{"jsonrpc":"2.0","id":1,"method":"initialize","params
   ↪  ":{"protocolVersion":"2025-06-18","capabilities":{},"clientInfo"
   ↪  :{"name":"t","version":"0"}}}' | algebraking
   ↪  mcp
   ```

3. Redo the registration: `claude mcp remove algebraking -s user` and then `./install.sh`.
4. Restart the Claude Code session.

Timeouts. `Error, (in series) computation timed out after 2 s`: the engine was restarted and the session continues. Increase it with `ALGEBRAKING_TIMEOUT=300`; if it is a heavy symbolic calculation, consider simplifying it by hand or splitting it.

Cache without permission. `Error, cannot create .../.cache/algebraking: Permission denied`: the engine is written to `$XDG_CACHE_HOME/algebraking` (or `~/.cache/algebraking`). Set `XDG_CACHE_HOME` to a writable directory.

Debugging. `ALGEBRAKING_DEBUG=1 algebraking -e '...'` shows the engine's `stderr` (`[engine] ...`) and the Python tracebacks of internal errors. To test the engine in isolation:

```
echo '{"id":1,"op":"ping"}' | python3 engine/engine.py
```

pip error "externally managed environment". `install.sh` already retries with `--break-system-packages` (only in the user directory). If you prefer, use a venv and point `ALGEBRAKING_PYTHON` to its Python, or `sudo apt install python3-sympy python3-mpmath`.

---

9 Validation

AlgebraKing reproduced **257/257** checks from the reference verification suite, coming from a research project (all 257 checks gave the same verdict).

Repository test suites (run to write this guide):

```
cargo test --release                        # 39 testes: 26 unitários + 7 de
↪  CLI/golden + 6 de MCP, todos passando
python3 -m unittest engine/test_engine.py  # Ran 287 tests ... OK
```

The "golden" tests (`tests/golden/*.mpl` and `*.out`) compare the binary's output with the expected output; the MCP ones exercise the handshake, the 8 tools, the timeout and the engine restart.